The Bellman-Ford algorithm

Warmup Questions

Q1 We know Dijkstra's algorithm computes $\delta(s, v)$ for every vertex $v \in V$.

How to find actual shortest paths ?

$\delta(s, v)$ → length of the shortest s to v path

Q2 If G is a DAG, where some edges may have negative weights.

Then Dijkstra's algorithm may not work in this case.

Q1

Answer:

While running Dijkstra, we maintain an array

$v.\pi$ = Parent of v in the Shortest Path.

When we Select a node $v \notin S$ with at least one edge from S for which

$$d'(v) = \min_{\substack{e=uv \\ u \in S}} d(u) + w_{uv}$$ is as small as possible

we also set $v.\pi = u$

Finally to get a path from S to a vertex t

(a) Start from t

(b) follow Parent Pointers backward

$t \rightarrow t.\pi \rightarrow [t.\pi].\pi \cdots \rightarrow S$

(c) Reverse the sequence

Single-Source Shortest Path Problem

I/p: Directed weighted graph G and a source vertex s.

O/p: For each $v \in V(G)$ Compute, length of a shortest s-v path in G.

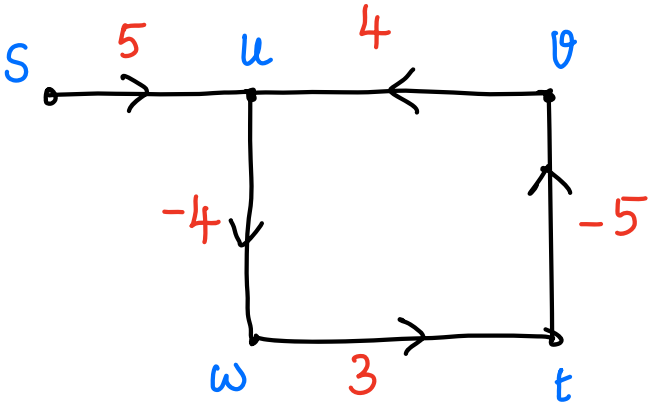
"length of path": Sum of edge weights of the path.

Dijkstra's Algorithm

Only works if G has no negative weight edges

Negative cycle or Negative weight cycle

Def: A cycle C in a edge weighted graph G is called negative cycle if $\sum_{e \in C} w_e < 0$



C: u w t v is a negative cycle

$$w(C) = \sum_{e \in C} w_e$$

$$= -4 + 3 - 5 + 4$$

$$= -2 < 0$$

Negative-weight cycles

If the graph contains a negative-weight cycle reachable from s, then shortest-path weights are not well defined.

We assume that

G may have negative edges

But NOT

Negative cycles

(Then also Dijkstra's algorithm doesn't work ☹)

Further Simplification

↓

Assume that input graph G has
No cycles (DAG)

↓

??

(Still Dijkstra's algo doesn't work)

We will first solve this Problem

We are interested in solving

Single-Source Shortest Path in DAG

- Edges are weighted
- Some edges may have negative weights

Any warm-up Ideas !!

HINT

① Topological ordering

② Optimal Substructure Property of Shortest Paths

TRUE | FALSE:

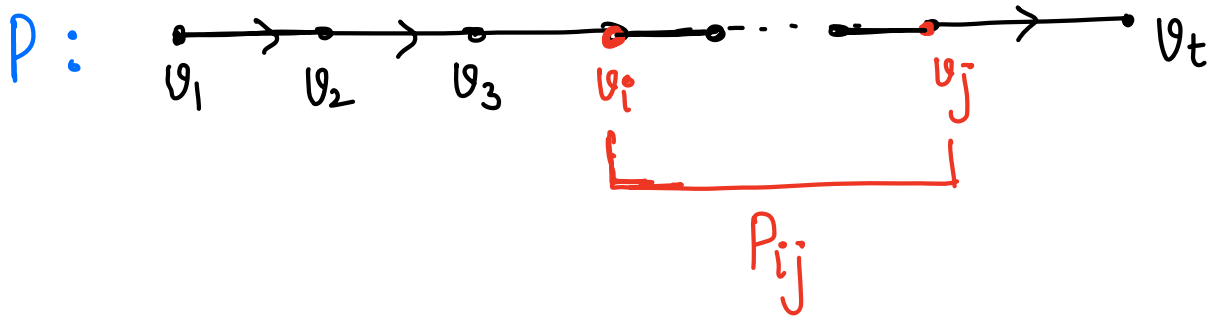
Let P be a shortest path from v_1 to v_t.

Then for any i and j, $1 \le i \le j \le t$

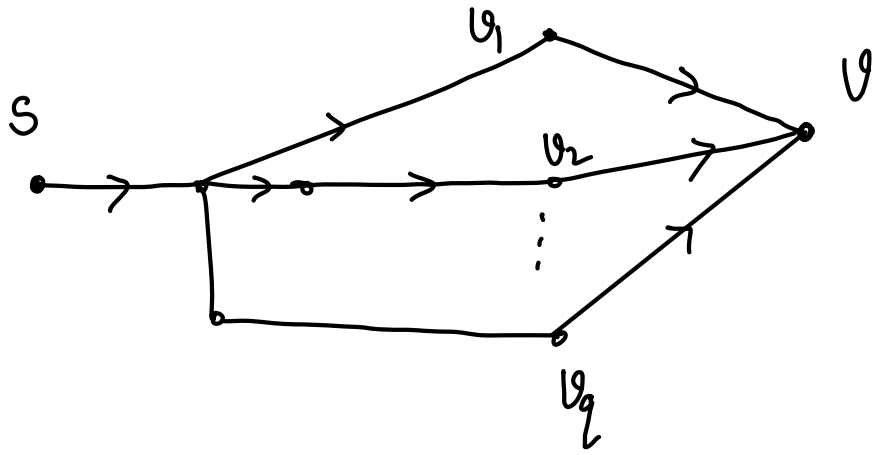
$P_{ij} = v_i, v_{i+1}, \ldots v_j$ be a subpath of P

from vertex v_i to v_j.

Then P_{ij} is a shortest path from v_i to v_j



Q:



If we know $\delta(s, v_1) \cdots \delta(s, v_q)$ then

how do you find $\delta(s, v)$?

Algorithm

DAG-SSP(s):

topologically Sort the vertices of G.

For each vertex $u \in V(G)$

If $u = s$

$d(u) = 0$

else

$d(u) = \infty$

For each vertex u, taken in topological order

For each vertex v adjacent to u $[u \to v]$

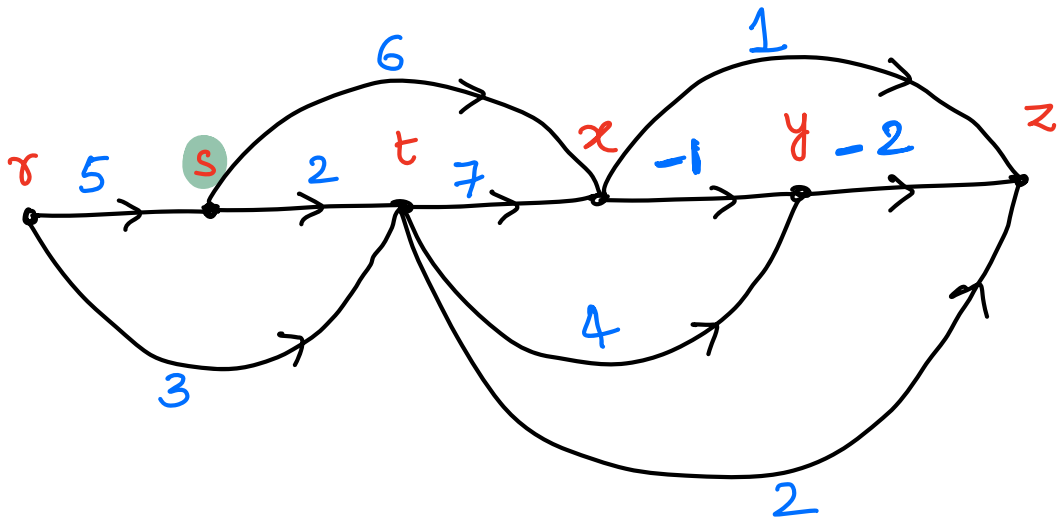
if $d(v) > d(u) + w_{uv}$

$d(v) = d(u) + w_{uv}$

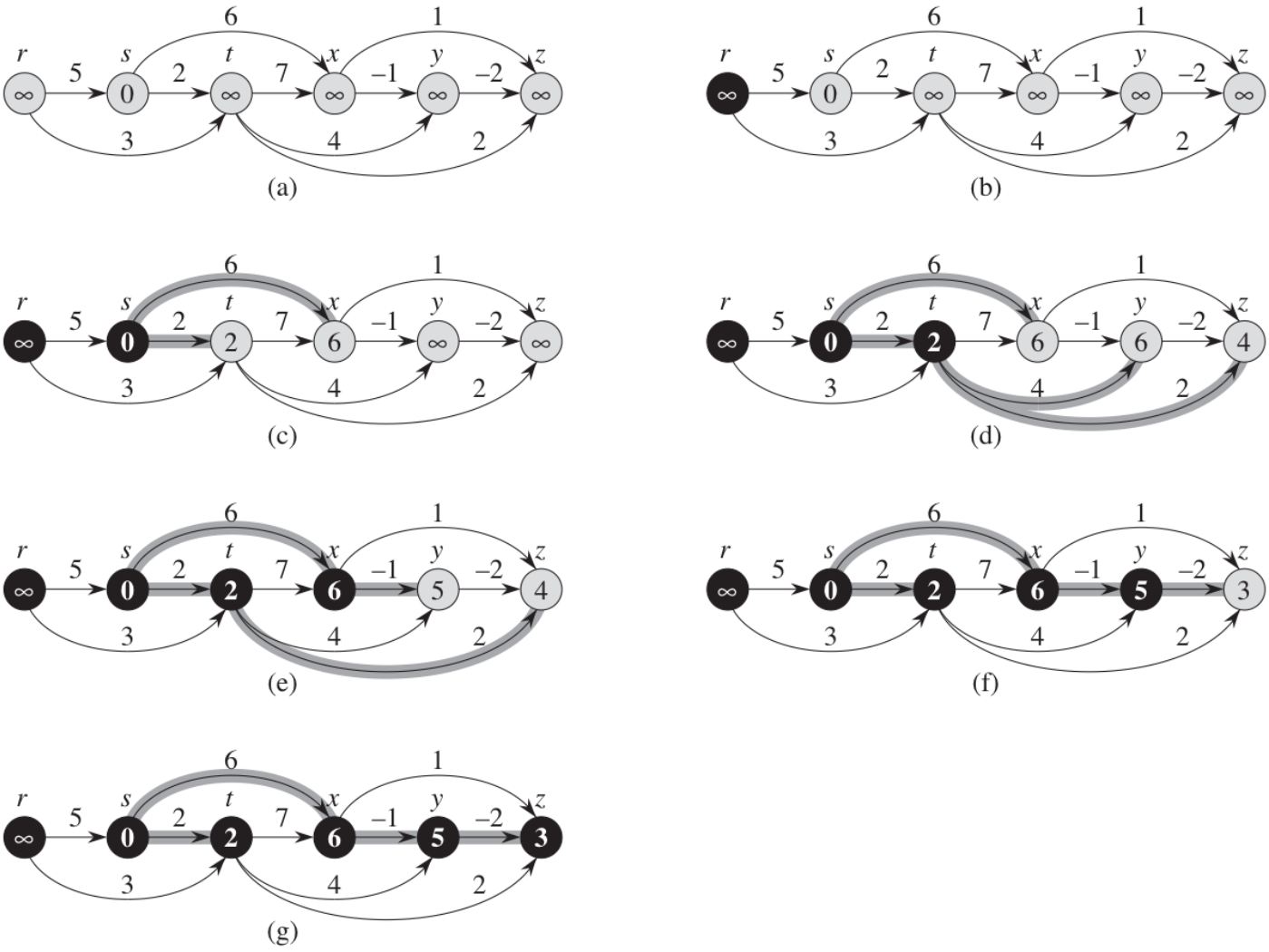
Running time: $\Theta(V+E)$

I/p: Adjacency list representation.

Example:



Ref: Coreman Chap 24



Back to the original Problem

- G may have negative edges

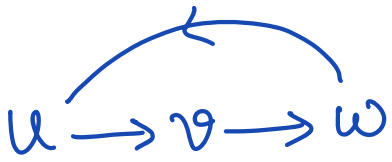
  But NOT

  Negative cycles

- G is NOT a DAG

Does the same idea work ??

Suppose the input graph contains a directed cycle



To compute $d(w)$ we need $d(v)$

" " $d(v)$ " " $d(u)$

" " $d(u)$ " " $d(w)$

We get stuck in an infinite loop ☹

We need New Ideas !!

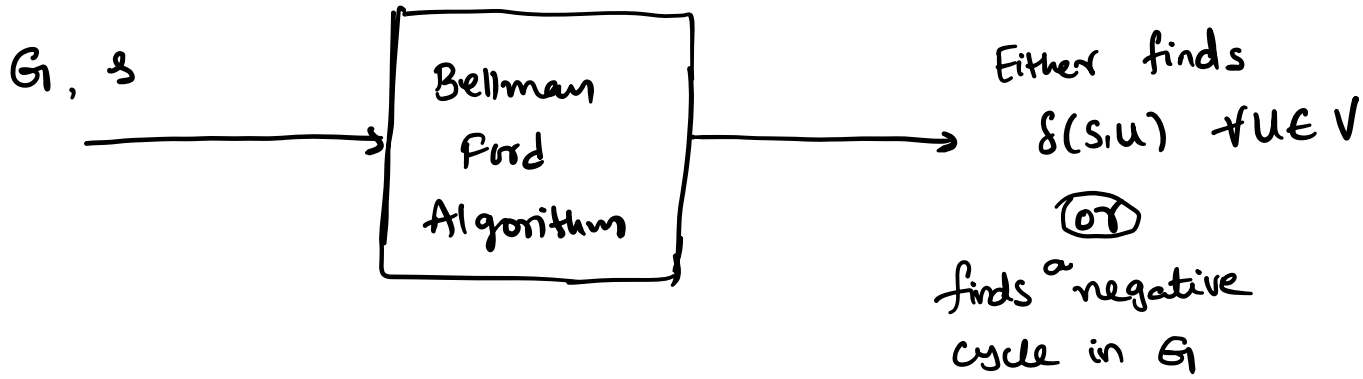
The Bellman-Ford algorithm

Ref: [KT-book]

The Algorithm Solves the Single-Source Shortest Paths Problem in which edge weights may be negative.

(*) If there is a negative-weight cycle that is reachable from the source, the algorithm indicates that no solution exists.

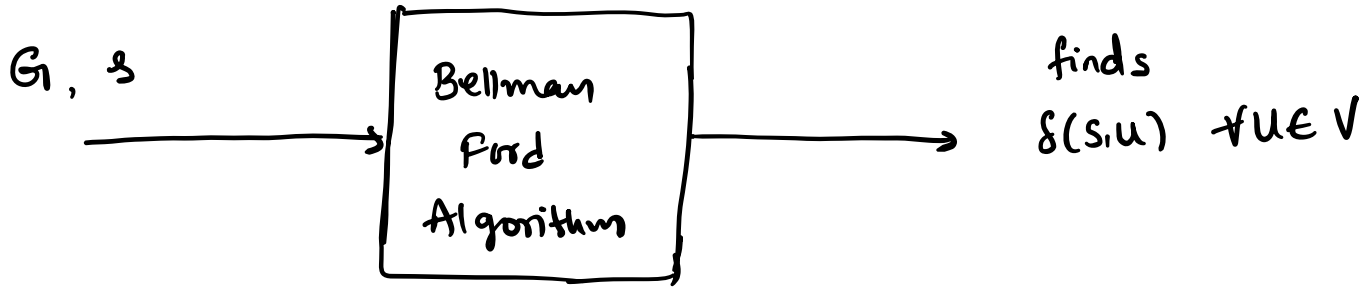
If there is no such cycle, the algorithm produces the shortest paths and their weights.



For Convenience*,

Assume that G has no negative cycles reachable from the source.

[Reason is given at the end of the algorithm]



if G has negative cycles

(*) Computing shortest cycle free s-t Path is NP-Complete

[more on this next week]

TRUE | FALSE: Suppose G has no negative cycles, then for every v, there is a shortest s-v path with at most $n-1$ edges.

Recall

Every Prefix of a Shortest Path is itself a Shortest Path.

Suppose the Shortest Path from s to t is

$$s \to v_1 \to v_2 \dots \to v_m \to t$$ then

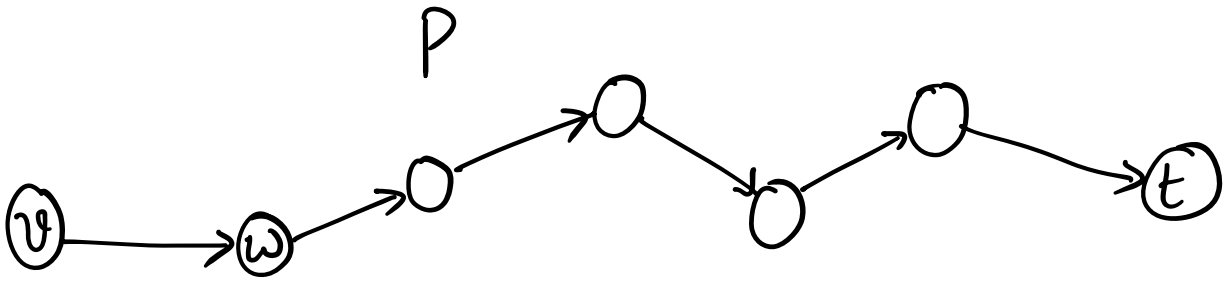
every Prefix

$s \to v_1 \to \dots \to v_r$ is a shortest Path to v_r

$r \in \{1, \dots m\}$

Slightly easier version of the Problem

Given s & t find Shortest s to t Path.

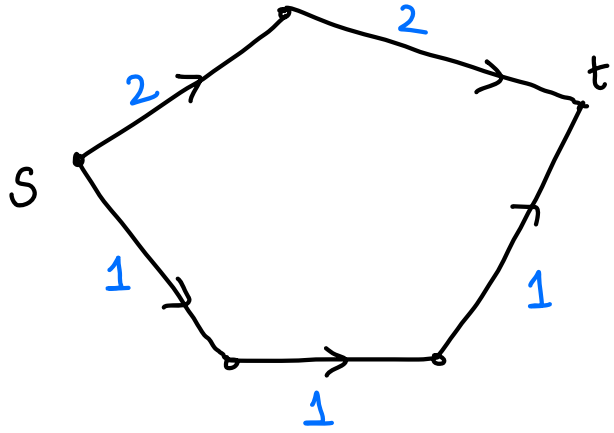


The minimum cost Path P from v to t using at most i edges.

Let $OPT[i, v]$ denote the minimum cost v-t Path using atmost i edges.

From lemma 1, our original Problem is to compute $OPT[n-1, s]$

Example



$OPT[2,S] = 4$

$OPT[3,S] = 3$

What is the Recurrence for OPT[i,v] ?

② [Optimal substructure]

How to express $OPT[i,v]$ using smaller problems.



Let P be an optimal path representing $OPT[i,v]$

① If P uses atmost $i-1$ edges then

$$OPT[i,v] = OPT[i-1,v]$$

② If P uses i edges, and the first edge is vw then

$$OPT[i,v] = w_{vw} + OPT[i-1, w]$$

$\therefore$ we get, for $i>0$,

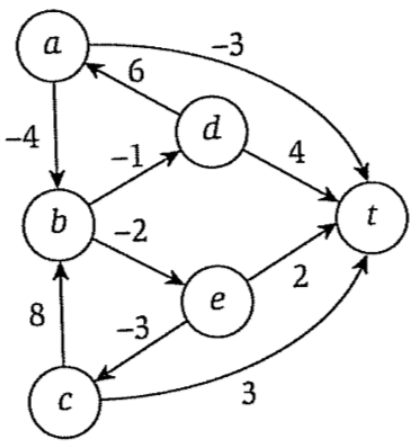
$$OPT[i,v] = \min\left\{ OPT[i-1,v],\ \min_{w \in V} \{OPT[i-1,w] + w_{vw}\} \right\} \quad \text{— ①}$$

Using this reccurence, we get the following DP algorithm to find $OPT[n-1, s]$

Finds the minimum cost of an s-t path

```
Shortest-Path(G, s, t)
  n = number of nodes in G
  Array M[0 ... n − 1, V]
  Define M[0, t] = 0 and M[0, v] = ∞ for all other v ∈ V
  For i = 1, ..., n − 1
    For v ∈ V in any order
      Compute M[i, v] using the recurrence (1)
    Endfor
  Endfor
  Return M[n − 1, s]
```

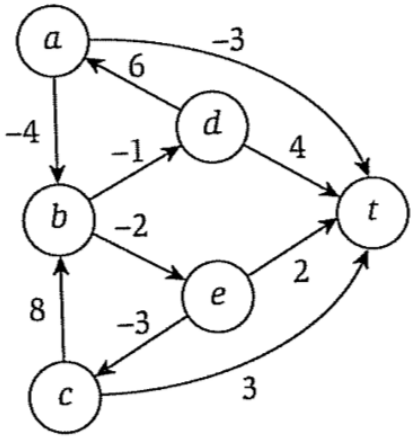
here we use M instead OPT



(a)

	0	1	2	3	4	5
t	0	0	0	0	0	0
a	∞	-3	-3			
b	∞	∞	0			
c	∞	3	3			
d	∞	4	3			
e	∞	2	0			

Example: Find a shortest path from each node to t.

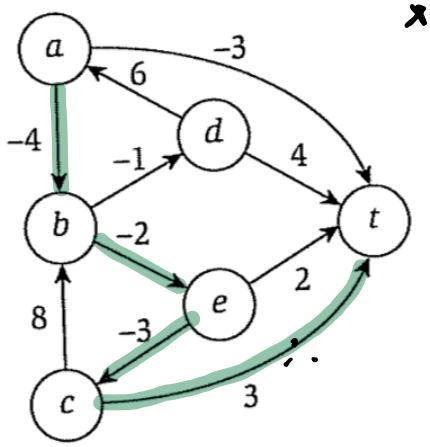


(a)

$M[i,v]$

	0	1	2	3	4	5
t	0	0	0	0	0	0
a	∞	−3	−3	−4	−6	−6
b	∞	∞	0	−2	−2	−2
c	∞	3	3	3	3	3
d	∞	4	3	3	2	0
e	∞	2	0	0	0	0

Table M



(a)

$a \rightsquigarrow t$

cost = $-4-2-3+3 = -6$

* A single row in the table corresponds to the shortest path from a particular node to t, as we allow the path to use an increasing number of edges.

* The last column indicates the cost of the shortest path from each node to t

Running time.

Table M has n^2 entries and each entry can take $O(n)$ time to complete, as there are at most n nodes $w \in V$ we have to consider.

$\therefore$ Algorithm runs in $O(n^3)$ time computes the minimum cost of an s-t path in any graph that has no negative cycles.

Finally

Q: How to check if G has a negative cycle using Bellman-Ford algorithm?

Theorem:

If there is no negative cycle in G

$$\Leftrightarrow$$

$$OPT[n, v] = OPT[n-1, v] \quad \text{for all } v.$$

Proof:

Forward:

$$OPT[n,v] \neq OPT[n-1,v] \quad \text{for some } v$$

let $OPT[n,v] = l$

let $P: v \rightsquigarrow t$ Path of length l

- if P uses at most $n-1$ edges then $OPT[n,v] = OPT[n-1,v]$ Which is a Contradiction.
- If P uses exactly n edges,

then P contains a cycle C

Because G has no negative cycles, the total weight of C is non-negative.

let P' be the path obtained from P by removing the cycle C. Then

P' is a $v \rightsquigarrow t$ Path having at most $n-1$ edges

that is $OPT[n,v] = OPT[n-1,v]$, Which is a Contradiction.

Backward:

Given $OPT[n,v] = OPT[n-1,v]$ for all $v \in V(G)$

let $d(v) = OPT[n,v] = OPT[n-1,v]$

let $C: v_1 v_2 \ldots v_k$ be an arbitrary cycle in G

Claim: $w(C) \geq 0$

Consider the edge $v_i v_{i+1}$ of C



$$OPT[n, v_i] \leq OPT[n-1, v_{i+1}] + w_{v_i v_{i+1}}$$

$$d(v_i) \leq d(v_{i+1}) + w_{v_i v_{i+1}}$$

$$w_{v_i v_{i+1}} \geq d(v_i) - d(v_{i+1})$$

Now $$w(C) = \sum_{i=1}^{k-1} w_{v_i v_{i+1}} + w_{v_k v_1}$$

$$= \sum_{i=1}^{k-1} \left[d(v_i) - d(v_{i+1}) \right] + d(v_k) - d(v)$$

[All terms gets cancelled]

$$\geq 0$$

BELLMAN-FORD(G, w, s) [Ref: Coreman et al.]

$d(s) = 0$

For each $v \in V(G) - \{s\}$

$d(v) = \infty$

$v.\pi = NIL$

For $i = 1$ to $n-1$

For each edge $uv \in E(G)$

if $d(v) > d(u) + w_{uv}$

$d(v) = d(u) + w_{uv}$

$v.\pi = u$

For each $uv \in E(G)$ } Checking for Negative cycle

if $d(v) > d(u) + w_{uv}$

return FALSE

return TRUE

Running time : $O(VE)$